Lecture et écriture dans un fichier

Les chemins de fichiers

Il y a deux façons de décrire l'emplacement d'un fichier : son chemin absolu ou son chemin relatif :

- Son **chemin absolu** décrit l'intégralité des dossiers (ou répertoires[1]) menant au fichier, peu importe l'endroit où on se trouven, depuis un répertoire dit « racine ». Sous Windows, la racine est le nom de volume (C:\, D:\…), sous les systèmes de la famille Unix, c'est « / ».
- Son **chemin relatif** décrit la succession de répertoires à parcourir en prenant comme point de départ le répertoire courrant dans lequel on se trouve.

Exemple : Quel est le chemin du fichier `explorer.exe` dans l'arborescence suivante ?

```
graph TD
A[C:] --> B;
A --> C[Program Files];
B[Windows] --> D{{explorer.exe}};
B --> E{{notepad.exe}};
A --> F["Program Files(x86)"];
A --> G[Users];
```

- Son chemin absolu est `C:\Windows\explorer.exe`.
- Son chemin relatif dépend du répertoire courant dans lequel on se trouve :

Répertoire courant	Chemin relatif
Windows	`explorer.exe`
C:	`Windows\explorer.exe`
Program Files	`..\Windows\explorer.exe`[2]

En Python, on peut déterminer le répertoire courant avec l'instruction `getcwd()` du module `os` (noter le double `\\` dans la chaîne de caractère pour 'échapper' le caractère `\`)

```
>>> import os
>>> os.getcwd()
'C:\\Program Files\\PyScripter'
```

Python reconnait indifféremment les chemins indiqués avec *backslash* (norme Windows) « `\` » qu'avec *slash* « `/` » (norme Unix et protocoles Internet).

f = open() et f.close()

Pour lire ou écrire dans un fichier depuis un programme Python, il faut d'abord **ouvrir le fichier**. On utilise la fonction `open()` qui prend pour paramètre le chemin (absolu ou relatif) du fichier et le mode d'ouverture :

- `'r'` pour ouvrir le fichier en mode lecture (*read-only*) est l'option par défaut.
- `'w'` en mode écrite (*write*), le contenu éventuel déjà présent dans le fichier est écrasé.
- `'a'` en mode ajout (*append*), ce que l'on écrit est ajouté à la fin du fichier.

On peut compléter le mode d'ouverture avec `b` pour un fichier binaire (image ou son par exemple), on obtient `rb'`, `'wb'`, `'ab'`.

⚠️ On ne peut pas ouvrir en mode lecture un fichier qui n'existe pas.

```
>>> f = open("fichier.txt ", "r")
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: 'fichier.txt'
```

Par contre en mode écriture ou ajout, si le fichier n'existe pas quand il est ouvert, alors il est créé.

```
f = open('fichier.txt', 'w')
```

Si `fichier.txt` ne se trouve pas dans le répertoire du programme Python, il faut donner son chemin pour y accéder avec `/` ou `\\`.

```
>>> f1 = open("C:/…/…/…/fichier1.txt ", "r")
>>> f2 = open("C:\\…\\…\\…\\fichier2.txt ", "r")
```

⚠️ Attention, il faut **toujours fermer un fichier après l'avoir ouvert**[3]. La méthode à utiliser est `close()`.

```
>>> f.close()
```

with open() as f:

Une autre façon de travailler sur un fichier est d'utiliser la structure suivante :

```
>>> with open("fichier.txt ", "w") as f:
        # bloc d'instructions
```

Dans ce cas-là, le fichier est automatiquement fermé à la fin du bloc d'instructions (attention à l'indentation), il n'y a pas besoin de le fermer (et cela limite les erreurs).

Écrire dans un fichier

Pour écrire dans un fichier, on utilise la méthode `write()` en lui passant en paramètre une **chaîne de caractères**[4] à écrire. `write()` renvoie le nombre de caractères qui ont été écrits dans le fichier[5].

f = open(...)

```
>>> f = open("fichier.txt", "w")
>>> f.write('France Paris\n')
13
>>> f.close()
```

with open(...) as f:

```
>>> with open("fichier.txt", "w") as f
        f.write('France Paris\n')
13
>>>
```

Noter le caractère « `\n` » pour indiquer un retour à la ligne. On peut ouvrir `fichier.txt` par exemple avec le blocnote et vérifier que le texte a bien été écrit.

On peut ensuite écrire deux autres lignes à la suite en mode `a`.

```
>>> f = open("fichier.txt", "a")
>>> f.write('Allemagne Berlin\nItalie Rome\n')
29
```

Si on ouvre `fichier.txt` dans le blocnote on constate que rien n'a été écrit ! En effet, il faut d'abord fermer le fichier.

```
>>> f.close()
```

Cette fois le fichier a bien été modifié.

Lire un fichier

Pour lire les données dans un fichier, on utilise la méthode `read()` qui renvoie l'intégralité du fichier dans une chaine de caractères

f = open(...)

```
>>> f = open("fichier.txt", "r")
>>> f.read()
'France Paris\nAllemagne Berlin\nItalie Rome\n'
>>> f.close()
```

with open(...) as f:

```
>>> with open("fichier.txt", "r") as f
    f.read()
'France Paris\nAllemagne Berlin\nItalie Rome\n'
>>>
```

qu'on peut afficher avec `print(f.read())` pour ne pas voir les retours à la ligne `\n`.

A noter, un fichier n'est lu qu'une fois avant d'être refermé.

f = open(...)

```
>>> f = open("fichier.txt", "r")
>>> print(f.read())
France Paris
Allemagne Berlin
Italie Rome

>>> print(f.read())

>>> f.close()
```

with open(...) as f:

```
>>> with open("fichier.txt", "r") as f
    print(f.read())
    print(f.read())
France Paris
Allemagne Berlin
Italie Rome
>>>
```

Après le premier `read()`, l'interpréteur Python est arrivé au bout du fichier. Il ne recommence pas à le lire depuis le début et rien n'apparait après le second `read()`. Pour recommencer au début du fichier, il faut le fermer et le rouvrir.

Au lieu de `.read()` qui lit tout le fichier, on peut ne lire qu'une seule ligne à la fois avec la méthode `readline()`

f = open(...)

```
>>> f = open("fichier.txt", "r")
>>> f.readline()
'France Paris\n'
>>> f.readline()
'Allemagne Berlin\n'
>>> f.close()
```

with open(...) as f:

```
>>> with open("fichier.txt", "r") as f
    f.readline()
    f.readline()
'France Paris\n'
'Allemagne Berlin\n'
>>>
```

ou itérer sur toutes les lignes avec `for… in …`.

f = open(...)

```
>>> f = open("fichier.txt", "r")
>>> for li in f:
        print(li)
France Paris

Allemagne Berlin

Italie Rome
```

```
>>> f.close()
```

with open(...) as f:

```
>>> with open("fichier.txt", "r") as f
    for li in f:
        print(li)
France Paris

Allemagne Berlin

Italie Rome
```

Ici, la variable `li` est une chaine de caractère qui prend la valeur de chaque ligne de `fichier.txt`.

On peut aussi mentionner la méthode `readlines()` qui permet lire l'intégralité d'un fichier dans une tableau (possible quand le fichier à lire n'est pas trop gros puisqu'il est copié intégralement dans une variable, c'est-à-dire dans la mémoire vive de l'ordinateur, il faut que la taille de celle-ci soit suffisante).

```
>>> with open("fichier.txt", "r") as f:
        f.readlines()
['France Paris\n', 'Allemagne Berlin\n', 'Italie Rome\n']
```

⚠️ Attention donc à ne pas confondre `readline()` qui renvoie une seule ligne dans une chaîne de caractères, avec `readlines()` qui renvoie un tableau de toutes les lignes.

---

1. Depuis Windows 7, le terme « dossier » remplace « répertoire » ↩
2. « `..` » désigne le répertoire parent. ↩
3. On peut essayer de supprimer un fichier ouvert depuis Windows explorer pour s'en convaincre. ↩
4. La méthode `write()` n'accepte en paramètre que des chaînes de caractères, pour écrire des nombres il faut les convertir en chaîne avant. ↩
5. On peut récupérer cette valeur par exemple pour vérifier que le fichier contient bien le texte qu'on y a écrit. ↩